ROADMAP DOCUMENT

Blood Reach BD

A Real-Time Blood Availability & Donor Matching Platform

Covering all 64 Districts & 7 Divisions of Bangladesh

Name	Md. Mehedi Hasan
Student ID	251035039
Program	B.Sc. in A.I and Data Science
Course	DBMS and SWE
Supervisor	Rakib Abdullah
Semester	Summer 2026

Technology Stack

Layer	Technology
Backend	Python 3.11, FastAPI, SQLAlchemy
Database	PostgreSQL 15
Frontend	HTML5, CSS3, Vanilla JavaScript
Auth	JWT, bcrypt
Version Control	Git, GitHub

"Connecting donors, seekers, and hospitals — one drop at a time."

Contents

1. Project Overview

Blood Reach BD is a full-stack web platform designed to solve critical blood availability challenges across Bangladesh. The system connects blood donors, seekers, and hospital administrators in real time through intelligent location-based matching, urgency prioritisation, and automated notifications.

Key Highlights

- Covers **all 64 districts and 7 divisions** of Bangladesh.
- Four user roles: DONOR SEEKER HOSPITAL ADMIN SUPERADMIN
- Built with **FastAPI** + **PostgreSQL** for a fast, scalable backend.
- **Location-based matching** using district/division data.
- **Urgency prioritisation** queues critical requests first.
- **Automated notifications** via email/SMS triggers.
- Estimated development timeline: **10 weeks**.

1.1. Mission Statement

To eliminate preventable deaths caused by blood unavailability in Bangladesh by providing a reliable, real-time digital bridge between donors and recipients — empowering hospitals and individuals alike with transparent, data-driven tools.

1.2. Problem Statement

- Patients in emergency situations cannot find matching donors quickly enough.
- Hospital blood banks lack real-time inventory visibility across the network.
- Existing solutions are fragmented, offline, or geographically limited.
- No centralised platform connects all 64 districts of Bangladesh under one system.

1.3. Proposed Solution

A three-tier web application (Backend API / PostgreSQL DB / HTML-JS Frontend) providing:

1. Instant donor–seeker matching filtered by blood group and district.
2. Real-time urgency queue for critical requests.
3. Hospital inventory management with low-stock alerts.
4. Role-based dashboards for all four user types.
5. Analytics for national blood supply monitoring.

2. User Roles

Badge	Role	Responsibilities
DONOR	Donor	Register availability, update blood group & location, respond to requests, view donation history.
SEEKER	Seeker	Post blood requests, browse nearby donors, mark requests as fulfilled.
HOSPITAL ADMIN	Hospital Admin	Manage hospital blood inventory, approve/reject requests, view stock reports.
SUPERADMIN	Superadmin	Full platform access: user management, analytics, system settings, audit logs.

3. Technology Stack

Layer	Technology	Purpose
Backend	`Python 3.12.7, FastAPI, SQLAlchemy`	REST API, ORM, async endpoints
Database	`PostgreSQL 18`	Relational data persistence
Frontend	`HTML5, CSS3, Vanilla JS`	Responsive UI without frameworks
Authentication	`JWT, bcrypt`	Stateless auth, password hashing
Version Control	`Git, GitHub`	Source control, collaboration

4. System Modules

#	Module	Description	Phase
1	**User Management**	Registration, login, profile management for all roles	P1–P2
2	**Donor Management**	Availability toggle, donation history, blood group CRUD	P2
3	**Blood Request System**	Create, update, track blood requests by seekers	P2
4	**Location-Based Matching**	Geo-filter donors by district/division, proximity ranking	P2–P3
5	**Urgency Prioritisation**	Queue requests by severity; critical flag escalation	P3
6	**Notification Engine**	Email/SMS alerts on match, fulfillment, low stock	P3
7	**Hospital Inventory Mgmt**	Real-time blood unit tracking, low-stock alerts	P3
8	**Analytics Dashboard**	National supply charts, district heatmaps, trends	P4
9	**Authentication & Security**	JWT tokens, bcrypt hashing, session management	P1–P2
10	**Role-Based Access Control**	Middleware guards per role, endpoint permissions	P2

5. Database Schema Overview

5.1. Core Entities

Table	Primary Key	Key Columns
users	user_id (UUID)	name, email, password_hash, role, district_id, created_at
donors	donor_id (UUID)	user_id (FK), blood_group, is_available, last_donated, total_donations
blood_requests	request_id (UUID)	seeker_id (FK), blood_group, quantity_units, urgency_level, status, district_id
hospitals	hospital_id (UUID)	name, district_id, admin_user_id (FK), contact_info
hospital_inventory	inv_id (UUID)	hospital_id (FK), blood_group, units_available, updated_at
districts	district_id (INT)	name, division_id (FK), latitude, longitude
divisions	division_id (INT)	name (one of 7 Bangladesh divisions)
notifications	notif_id (UUID)	recipient_id (FK), type, message, is_read, sent_at
audit_logs	log_id (UUID)	actor_id (FK), action, entity_type, entity_id, timestamp

5.2. Key Relationships

- `users` → `donors` (1:1) — every donor is a user.
- `users` → `blood_requests` (1:N) — seekers post multiple requests.
- `hospitals` → `hospital_inventory` (1:N) — one hospital, multiple blood groups.
- `districts` → `divisions` (N:1) — 64 districts in 7 divisions.
- `notifications` → `users` (N:1) — many alerts per user.

6. Weekly Roadmap

6.1. Timeline Summary

Week	Phase	Focus	Colour
1	**Planning**	Problem analysis, SRS, DFD, GitHub setup	
2–3	**Backend I**	Project skeleton, DB design, auth endpoints	
4–5	**Backend II**	Core modules: donor, requests, matching	
6	**Backend III**	Notifications, urgency engine, RBAC middleware	
7	**Frontend I**	HTML templates, CSS design system, JS interactions	
8	**Integration**	Frontend ↔ API wiring, testing flows	
9	**Testing**	Unit tests, integration tests, bug fixes	
10	**Deployment**	Final docs, demo, GitHub release	

Week 1 | Planning & Documentation

Goal: Lay the complete analytical and documentary foundation for the project before writing a single line of code. Establish team norms, tooling conventions, and a shared understanding of scope.

Deliverables

- Software Requirements Specification (SRS)
- Data Flow Diagram — Level 0 (Context Diagram)
- Data Flow Diagram — Level 1 (System Processes)
- User Roles & Permissions Matrix
- Entity Relationship Diagram (ERD) draft
- GitHub repository with `README.md`, `.gitignore`, and branch strategy
- Project folder structure proposal

Tools Used

- **draw.io** — DFD & ERD diagrams
- **Google Docs / LaTeX** — SRS document
- **GitHub** — Repository, Issues, Project board
- **Notion / Trello** — Task management

No Coding This Week

All effort directed to analysis and documentation. Clean, thorough planning prevents expensive rework in later weeks.

Weeks 2–3 | Backend Phase I — Foundation

Goal: Establish the full project skeleton, configure the database, and ship working authentication endpoints.

Deliverables

- FastAPI project initialised with virtual environment
- PostgreSQL schema migrated (Alembic)
- `/auth/register` and `/auth/login` endpoints (JWT)
- Password hashing with bcrypt
- Role middleware skeleton (Donor, Seeker, Hospital, Superadmin)
- Environment config via `.env` and Pydantic Settings
- Postman collection for auth routes

Tools Used

- **Python 3.12.7** + **FastAPI**
- **SQLAlchemy 2.x** ORM
- **Alembic** — DB migrations
- **PostgreSQL 15** (local Docker)
- **bcrypt** + **python-jose** (JWT)
- **Postman** — API testing
- **Git** — feature branch workflow

Weeks 4–5 | Backend Phase II — Core Modules

Goal: Implement the three primary business modules: Donor Management, Blood Request System, and Location-Based Matching.

Deliverables

- Donor CRUD endpoints (availability toggle, blood group update)
- Donation history tracking
- Blood request creation, update, status tracking
- District/division seeding (64 districts, 7 divisions)
- Location-based donor search endpoint (filter by district + blood group)
- Proximity ranking logic
- Unit tests for all new endpoints

Tools Used

- **FastAPI** routers (modular)
- **SQLAlchemy** queries + joins
- **pytest** + **httpx** — unit testing
- **pgAdmin 4** — DB inspection
- **GitHub** — PR reviews

Week 6 | Backend Phase III — Notifications, Urgency & RBAC

Goal: Complete the remaining backend modules: urgency prioritisation engine, automated notification system, and full RBAC enforcement.

Deliverables

- Urgency queue: critical / high / normal levels
- Escalation logic for unmatched critical requests (>30 min)
- Email notification service (registration, match, fulfillment)
- Notification model + mark-as-read endpoint
- Hospital inventory CRUD + low-stock alert trigger
- Role guards enforced on all protected routes
- Audit log middleware for sensitive actions

Tools Used

- **FastAPI BackgroundTasks**
- **smtplib / Mailgun API**
- **APScheduler** — escalation cron
- **SQLAlchemy** — inventory queries
- **pytest** — integration tests

Week 7 | Frontend Phase — UI Design & JavaScript

Goal: Build all HTML pages, establish the CSS design system, and wire up interactive JavaScript components.

Deliverables

- Design system: CSS variables, typography, colour tokens
- Pages: Landing, Login/Register, Donor Dashboard, Seeker Dashboard, Hospital Admin Panel, Superadmin Console
- Blood request submission form with live validation
- Donor availability toggle UI
- Notification bell component
- Responsive mobile layout (media queries)

Tools Used

- **HTML5** semantics
- **CSS3** custom properties, Flexbox, Grid
- **Vanilla JavaScript** (ES6+)
- **Fetch API** — async HTTP
- **Figma** (wireframes reference)
- **Chrome DevTools**

Week 8 | Integration — Frontend meets Backend

Goal: Connect all frontend pages to live FastAPI endpoints. Resolve CORS, authentication state, and data flow issues.

Deliverables

- JWT stored in `localStorage`; auth headers on all requests
- CORS configured in FastAPI for the frontend origin
- Full donor registration → login → availability toggle flow
- Full seeker request → matching → notification flow
- Hospital inventory view + update wired
- Superadmin user list and analytics charts (Chart.js)
- End-to-end smoke tests for all role journeys

Tools Used

- **Fetch API** + **async/await**
- **Chart.js** — analytics visualisation
- **FastAPI CORS Middleware**
- **Postman** — contract verification
- **Git** — integration branch

Week 9 | Testing, QA & Bug Fixes

Goal: Achieve comprehensive test coverage, resolve all critical bugs, and harden the platform for deployment.

Deliverables

- Unit tests for all API routes (≥80% coverage)
- Integration tests for role-based workflows
- Manual QA checklist completed for all 4 roles
- Security review: SQL injection, XSS, IDOR checks
- Performance test: 100 concurrent requests on matching endpoint
- Bug tracker cleared (P0 & P1 issues resolved)
- `CHANGELOG.md` updated

Tools Used

- **pytest** + **pytest-cov**
- **httpx** — async test client
- **Locust** — load testing
- **OWASP ZAP** — security scan
- **GitHub Issues** — bug tracking

Week 10 | Deployment, Documentation & Submission

Goal: Deploy the application, finalise all documentation, record a demo, and submit the project.

Deliverables

- Production deployment (Railway / Render / VPS)
- PostgreSQL production database provisioned
- `README.md` polished with setup instructions
- API documentation via FastAPI `/docs` (Swagger UI)
- Demo video ($\leq$10 min, all role journeys shown)
- GitHub Release v1.0.0 tagged
- Project report / LaTeX document submitted
- Live URL shared with supervisor

Tools Used

- **Railway** or **Render** — PaaS hosting
- **GitHub Actions** — CI/CD pipeline
- **OBS Studio** — demo recording
- **Swagger UI** — auto API docs
- **LaTeX** — final report typesetting

7. Project Risks & Mitigations

Risk	Severity	Mitigation Strategy
Scope creep	High	Strict module freeze after Week 2; change requests deferred to v2.
Notification delivery failures	Med	Retry queue with exponential back-off; fallback SMS via bKash API.
DB performance at scale	Med	Index on district_id, blood_group, urgency; query profiling in Week 9.
Auth token leakage	High	Short-lived tokens (15 min), refresh token rotation, HTTPS enforced.
Time overrun	Med	Weekly milestone checkpoints; MVP scope defined for weeks 1–6.

8. Success Metrics

- All 10 modules functional end-to-end.
- API response time < 300ms (p95).
- Test coverage $\geq 80\%$ on backend.
- Zero P0 (critical) bugs at submission.
- All 4 role dashboards fully operational.
- District data for all 64 districts seeded.
- Notification delivery rate $\geq 95\%$.
- Live deployment accessible via public URL.
- Demo video approved by supervisor.
- GitHub repository public and well-documented.